

Ex: perceptron

- Find the weights required to perform the following classification using perceptron network. Assume learning rate as 1 and initial weights as 0.
- Let w_1 w_2 w_3 w_4 $b = 0$ and the learning rate $a = 1$. Since the threshold = 0.2,

Input					Target (t)
x_1	x_2	x_3	x_4	b	
1	1	1	1	1	1
-1	1	-1	-1	1	1
1	1	1	-1	1	-1
1	-1	-1	1	1	-1

Lec 5

ADAPTIVE LINEAR NEURON (ADALINE)
&
MADALINE

Dr. Mohammad Mhawish

- The units with linear activation function are called linear units.
- A network with a single linear unit is called an *Adaline* (adaptive linear neuron).
- Adaline uses **bipolar activation for its input signals and its target output.**
- The weights between the input and the output are adjustable. The bias in Adaline acts like an adjustable weight, whose connection is from a unit with activations being always 1.
- Adaline is a net which **has only one output unit.**
- The Adaline network may be trained using **delta rule**. This learning rule is found to minimize the mean-squared error between the activation and the target value.

• *ADALINE* employs the **identity activation function** at the output unit **during training**. This implies that during training $y_{out} = y_{in}$.

• **During application** the following bipolar step function is used for activation.

$$y = \begin{cases} 1, & \text{if } y_{in} \geq 0 \\ -1, & \text{if } y_{in} < 0 \end{cases}$$

Learning by an *ADALINE* net is sensitive to the value of the learning rate. A too large learning rate may prevent the learning process to converge. On the other hand, if the learning rate is too low, the learning process is extremely slow. Usually, the learning rate is set on the basis of the inequality $.1 \leq m \times h \leq 1.0$, where m is the number of input units.

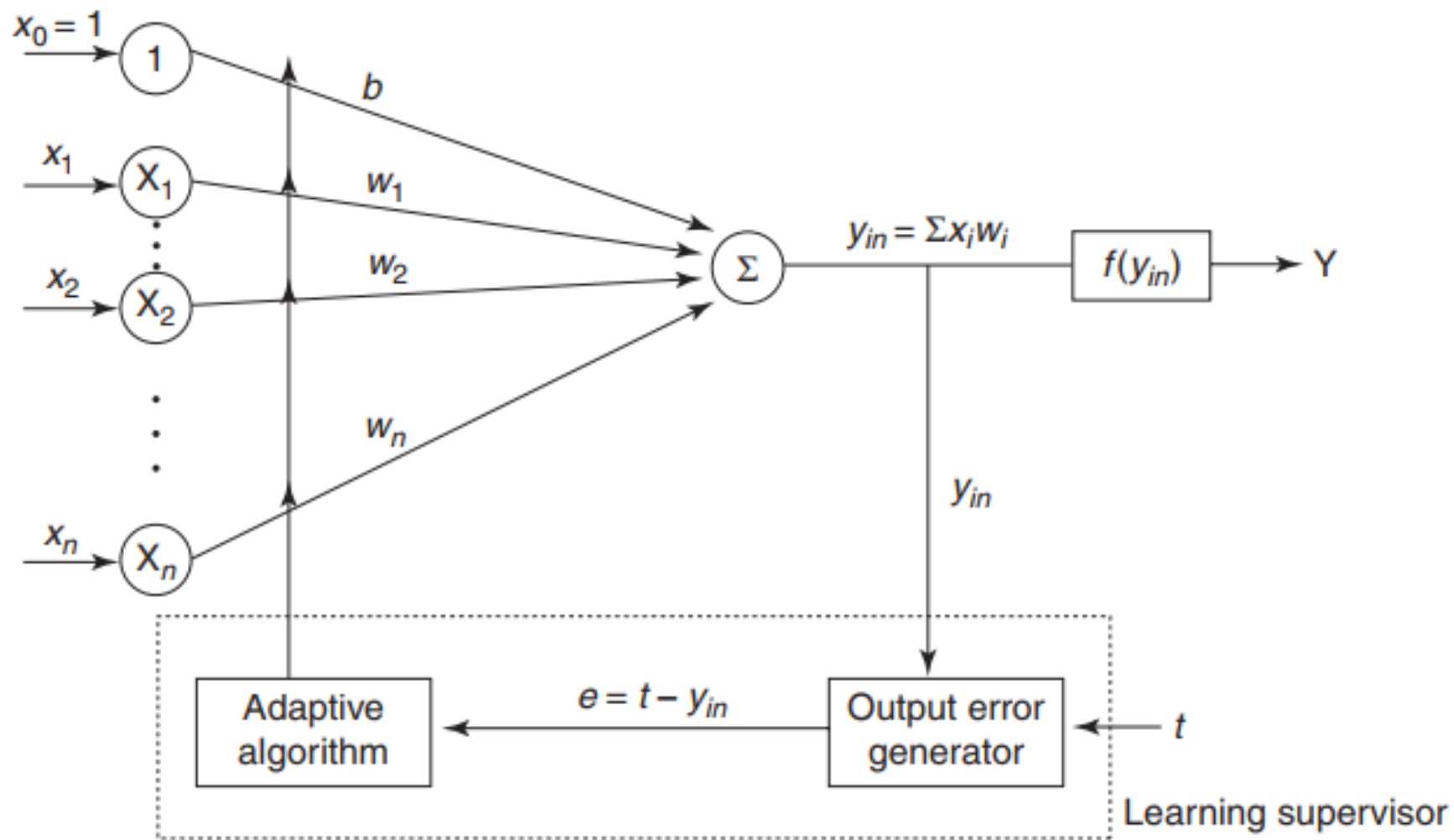
Delta Rule for Single Output Unit

- The delta rule updates the weights between the connections so as to **minimize the difference between the net input to the output unit and the target value.**
- The major aim is to minimize the error over all training patterns. This is done by reducing the error for each pattern, one at a time. The delta rule for adjusting the weight of I th pattern ($i = 1$ to n) is

$$\Delta w_i = \alpha (t - y_{in}) x_i$$

where Δw_i is the weight change; α the learning rate;

- The basic Adaline model consists of trainable weights. Inputs are either of the two values (+1 or -1) and the weights have signs (positive or negative).
- Initially, random weights are assigned.
- The net input calculated is applied to a quantizer transfer function (possibly activation function) that restores the output to +1 or -1.
- The Adaline model compares the actual output with the target output and on the basis of the training algorithm, the weights are adjusted.



Training Algorithm

Step 0: Weights and bias are set to some random values but not zero. Set the learning rate parameter α .

Step 1: Perform Steps 2–6 when stopping condition is false.

Step 2: Perform Steps 3–5 for each bipolar training pair $s:t$.

Step 3: Set activations for input units $i = 1$ to n .

$$x_i = s_i$$

Step 4: Calculate the net input to the output unit.

$$y_{in} = b + \sum_{i=1}^n x_i w_i$$

Step 5: Update the weights and bias for $i = 1$ to n :

$$w_i(\text{new}) = w_i(\text{old}) + \alpha (t - y_{in}) x_i$$

$$b(\text{new}) = b(\text{old}) + \alpha (t - y_{in})$$

Step 6: If the highest weight change that occurred during training is smaller than a specified tolerance then stop the training process, else continue. This is the test for stopping condition of a network.

$$E = (t - y_{in})^2$$

Testing Algorithm

- It is essential to perform the testing of a network that has been trained. When training is completed, the Adaline can be used to classify input patterns. A step function is used to test the performance of the network. The testing procedure for the Adaline network is as follows:

Step 0: Initialize the weights. (The weights are obtained from the training algorithm.)

Step 1: Perform Steps 2–4 for each bipolar input vector x .

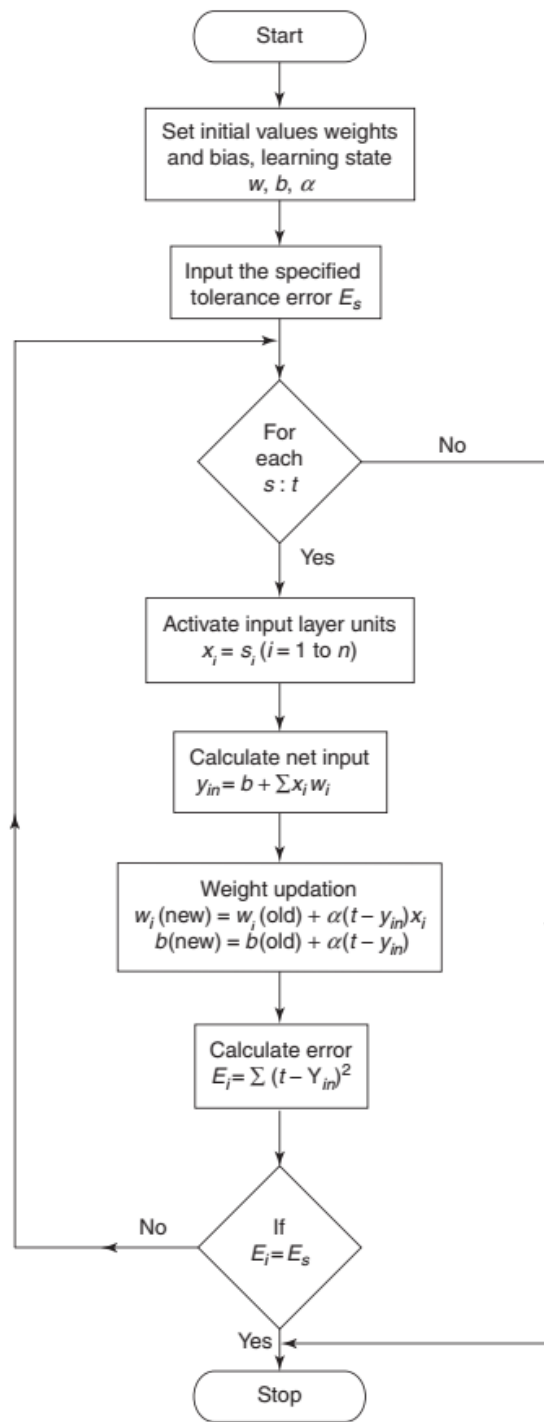
Step 2: Set the activations of the input units to x .

Step 3: Calculate the net input to the output unit:

$$y_{in} = b + \sum x_i w_i$$

Step 4: Apply the activation function over the net input calculated:

$$y = \begin{cases} 1 & \text{if } y_{in} \geq \theta \\ -1 & \text{if } y_{in} < \theta \end{cases}$$

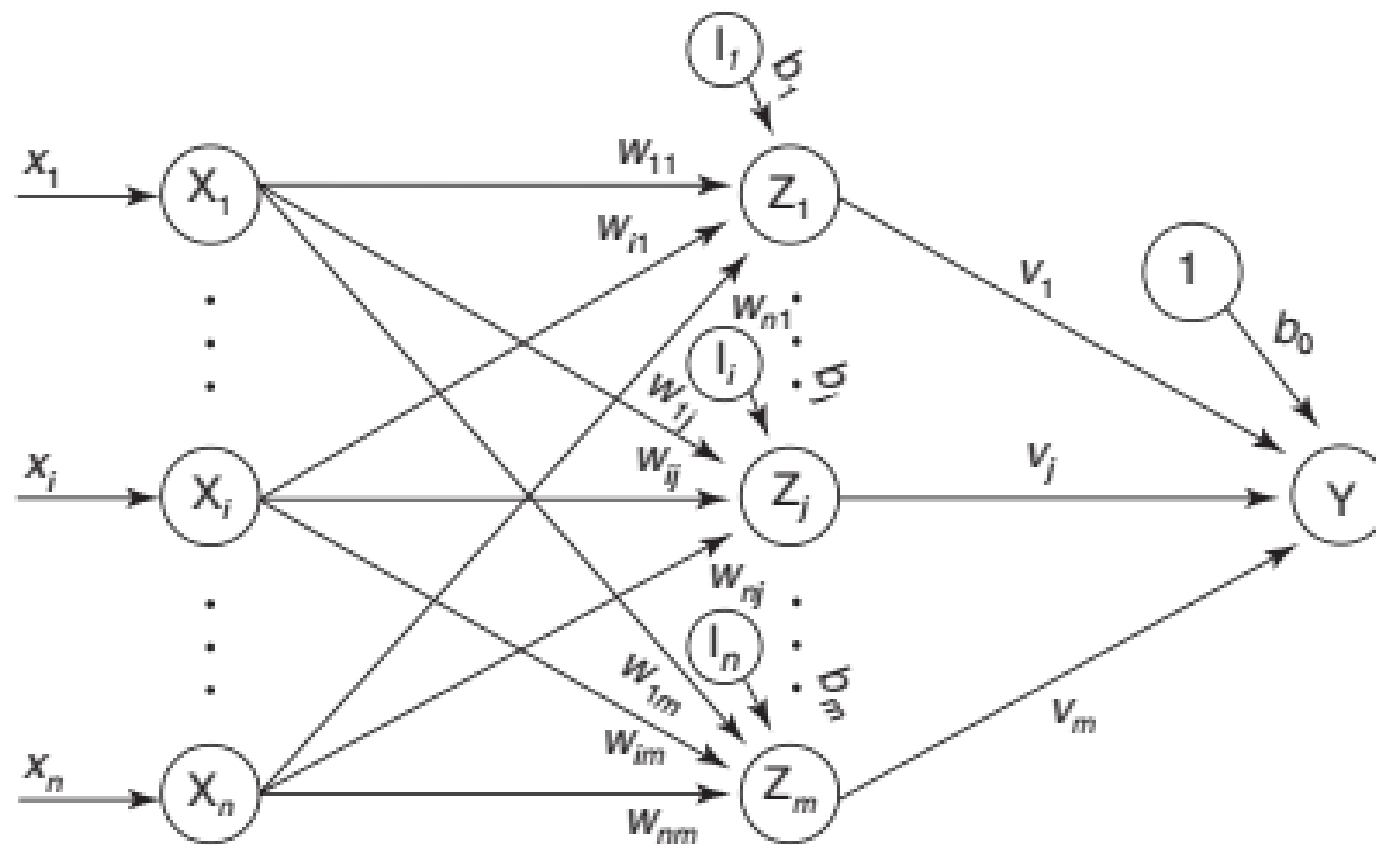


- Use Adaline network to train ANDNOT function with bipolar inputs and targets. Perform 2 epochs of training .
- Initially the weights and bias have assumed a random value say 0.2. The learning rate is also set to 0.2. The weights are calculated until the least mean square error is obtained. $E = (t - y_{in})^2$
- The initial weights are $w_1 = w_2 = b = 0.2$, and $a = 0.2$. For the first input sample $x_1 = 1, x_2 = 1, t = -1$, we calculate the net input as

MADALINE MULTIPLE ADAPTIVE LINEAR NEURONS

- The multiple adaptive linear neurons (Madaline) model **consists of many Adalines in parallel with a single output unit whose value is based on certain selection rules.**
- The weights that are connected **from the Adaline layer to the Madaline layer are fixed**, positive and possess equal values.
- The **weights** between the **input layer and the Adaline layer** are **adjusted** during the training process.
- The Adaline and Madaline layer neurons have a bias of excitation "1" connected to them. The training process for a Madaline system is similar to that of an Adaline.

- The Adaline layer is present between the input layer and the Madaline (output) layer; hence, the **Adaline layer can be considered a hidden layer**.
- The use of the hidden layer gives the net computational capability which is not found in single-layer nets, but this complicates the training process to some extent.



Training Algorithm

- In this training algorithm, only the weights between the hidden layer and the input layer are adjusted, and the weights for the output units are fixed.
- The weights $v_1, v_2, \dots, v_m$ and the bias b_0 that enter into output unit Y are determined so that the response of unit Y is 1. Thus, the weights entering Y unit may be taken as

$$v_1 = v_2 = \dots = v_m = \frac{1}{2}$$

$$b_0 = \frac{1}{2}$$

The activation for the Adaline (hidden) and Madaline (output) units is given by

$$f(x) = \begin{cases} 1 & \text{if } x \geq 0 \\ -1 & \text{if } x < 0 \end{cases}$$

Step 0: Initialize the weights. The weights entering the output unit are set as above. Set initial small random values for Adaline weights. Also set initial learning rate α .

Step 1: When stopping condition is false, perform Steps 2–3.

Step 2: For each bipolar training pair $s:t$, perform Steps 3–7.

Step 3: Activate input layer units. For $i = 1$ to n ,

$$x_i = s_i$$

Step 4: Calculate net input to each hidden Adaline unit:

$$z_{inj} = b_j + \sum_{i=1}^n x_i w_{ij}, \quad j = 1 \text{ to } m$$

Step 5: Calculate output of each hidden unit:

$$z_j = f(z_{inj})$$

$$= \begin{cases} 1, & \text{if } z_{inj} \geq 0 \\ -1, & \text{if } z_{inj} < 0. \end{cases}$$

Step 6: Find the output of the net:

$$y_{in} = b_0 + \sum_{j=1}^m z_j v_j$$

$$y = f(y_{in})$$

$$= \begin{cases} 1, & \text{if } y_{in} \geq 0 \\ -1, & \text{if } y_{in} < 0. \end{cases}$$

1. The weights should be adjusted only when there is a mismatch between the actual output y_{out} and the target output t .
2. The adjustments of the weights are to be done in a manner so that the possibility of producing the target output is enhanced.

Step 7: Calculate the error and update the weights.

1. If $t = y$, no weight updation is required.
2. If $t \neq y$ and $t = +1$, update weights on z_j , where net input is closest to 0 (zero):

$$b_j(\text{new}) = b_j(\text{old}) + \alpha (1 - z_{inj})$$

$$w_{ij}(\text{new}) = w_{ij}(\text{old}) + \alpha (1 - z_{inj})x_i$$

3. If $t \neq y$ and $t = -1$, update weights on units z_k whose net input is positive:

$$w_{ik}(\text{new}) = w_{ik}(\text{old}) + \alpha (-1 - z_{ink})x_i$$

$$b_k(\text{new}) = b_k(\text{old}) + \alpha (-1 - z_{ink})$$

Step 8: Test for the stopping condition. (If there is no weight change or weight reaches a satisfactory level, or if a specified maximum number of iterations of weight updation have been performed then stop, or else continue).

